ETAPAS DE PRÉ-PROCESSAMENTO

TRATAMENTO DE DADOS AUSENTES

Cristiane Neri Nobre

Tratamento de dados ausentes

- Base de dados podem contem dados ausentes e isto apresenta dificuldades relacionadas à **qualidade dos dados**.

- Dados ausentes podem ser causadas por:
  - problemas nos equipamentos que realizam a coleta
  - a transmissão e o armazenamento dos dados
  - ou problemas no preenchimento ou na entrados dos dados por seres humanos

- Algumas técnicas de AM conseguem lidar bem com dados ausentes, mas isso não acontece com todos os métodos de AM

Tratamento de dados ausentes

Idade	Sexo	Peso	Manchas	Temp.	# Int.	Diagnóstico
—	M	79	—	38,0	—	Doente
18	F	67	Inexistentes	39,5	4	Doente
49	M	92	Espalhadas	38,0	2	Saudável
18	—	43	Inexistentes	38,5	8	Doente
21	F	52	Uniformes	37,6	1	Saudável
22	F	72	Inexistentes	38,0	3	Doente
—	F	87	Espalhadas	39,0	6	Doente
34	M	67	Uniformes	38,4	2	Saudável

Int = Número de internações

Quais as razões para dados ausentes?

- O atributo não foi considerado importante na época da coleta (campo e-mail em 1990)
- Falta de conhecimento do campo (ex: tipo sanguíneo)
- Distração
- Falta de necessidade ou obrigação de apresentar um valor para o atributo (Ex. renda)
- Inexistência de um valor para o atributo em algumas instâncias (ex: número de partos de for homem)
- Problema com equipamento ou processo utilizado para coleta, transmissão e armazenamento de dados

Tratamento de dados ausentes

Quais as alternativas para se resolver este problema de dados ausentes?

1. Eliminar as instâncias com dados ausentes
2. Definir e preencher manualmente valores para os atributos com valores ausentes
3. Utilizar algum método ou heurística para automaticamente definir valores para atributos com valores ausentes
4. Empregar algoritmos de AM que lidam internamente com valores ausentes

Tratamento de dados ausentes

1. Eliminar as instâncias com dados ausentes

Esta alternativa é geralmente empregada quando um dos atributos com valores ausentes de uma instância é o que indica a sua classe.

Esta estratégia não é recomendada quando:

- Quando poucos atributos da instância possuem valores ausentes
- Quando o número de atributos com valores ausentes varia muito entre as instâncias com esse problema
- Quando o número de instâncias que restarem for pequeno

Tratamento de dados ausentes

2. Definir e preencher manualmente valores para os atributos com valores ausentes

Esta estratégia não é recomendada quando:

- O número de instâncias ou atributos com valores ausentes for muito grande

Tratamento de dados ausentes

3. Utilizar algum método ou heurística para automaticamente definir valores para atributos com valores ausentes

Esta estratégia é uma das mais recomendadas

Como imputar estes dados ausentes?

- Moda, média dos valores dos atributos
- Empregar um indutor para estimar o valor do atributo

Tratamento de dados ausentes

Exemplo:

Idade	Sexo	Peso	Manchas	Temp.	# Int.	Diagnóstico
—	M	79	—	38,0	—	Doente
18	F	67	Inexistentes	39,5	4	Doente
49	M	92	Espalhadas	38,0	2	Saudável
18	—	43	Inexistentes	38,5	8	Doente
21	F	52	Uniformes	37,6	1	Saudável
22	F	72	Inexistentes	38,0	3	Doente
—	F	87	Espalhadas	39,0	6	Doente
34	M	67	Uniformes	38,4	2	Saudável



Após a imputação pela média/moda

Idade	Sexo	Peso	Manchas	Temp.	# Int.	Diagnóstico
27	M	79	**Inexistentes**	38,0	**4**	Doente
18	F	67	Inexistentes	39,5	4	Doente
49	M	92	Espalhadas	38,0	2	Saudável
18	**F**	43	Inexistentes	38,5	8	Doente
21	F	52	Uniformes	37,6	1	Saudável
22	F	72	Inexistentes	38,0	3	Doente
27	F	87	Espalhadas	39,0	6	Doente
34	M	67	Uniformes	38,4	2	Saudável

Tratamento de dados ausentes

4. Empregar algoritmos de AM que lidam internamente com valores ausentes

Este é o caso, por exemplo, de alguns algoritmos indutores de árvore de decisão.

Em **Python**, veja os métodos:

Missforest - pip install missingpy

```
from missingpy import MissForest
```

KNNimputer -

```
from sklearn.impute import KNNImputer
```

Tratamento de dados ausentes

Na prática, em Python temos:

```
from sklearn.impute import KNNImputer

imputer = KNNImputer(n_neighbors=5)

 # Ajusta (aprende) os parâmetros SOMENTE com o treino
imputer.fit(X_train)



# Aplica os mesmos parâmetros aprendidos em ambos os conjuntos
X_train_imputed = imputer.transform(X_train)
X_test_imputed = imputer.transform(X_test)
```

Tratamento de dados ausentes

Vamos compreender a razão de usar a função fit apenas no treino e usar o predict no treino e teste?

- **Cenário**
- Imagine que você já fez o split treino/teste, e está imputando Age usando Fare como referência de similaridade (KNN com k=1).

Treino (6 passageiros, tudo conhecido):

ID	Age	Fare	Survived
T1	22	7.20	0
T2	38	71.30	1
T3	26	7.90	1
T4	35	53.10	1
T5	54	51.90	0
T6	2	21.10	1

Teste (2 passageiros — dados "nunca vistos"):

ID	Age	Fare	Survived
Te1	**NaN**	8.00	0
Te2	45	8.05	1

Tratamento de dados ausentes

- **Cenário B (correto): fit só no treino, transform em ambos**
- O "pool de vizinhos" disponível para imputar é **só o treino**. Calculando a distância de Te1 (Fare=8.00) até cada um:

Treino (6 passageiros, tudo conhecido):

ID	Age	Fare	Survived
T1	22	7.20	0
T2	38	71.30	1
T3	26	7.90	1
T4	35	53.10	1
T5	54	51.90	0
T6	2	21.10	1

Teste (2 passageiros — dados "nunca vistos"):

ID	Age	Fare	Survived
Te1	**NaN**	8.00	0
Te2	45	8.05	1

Vizinho candidato	|Fare − 8.00|
T1 (7.20)	0.80
T3 (7.90)	**0.10** ← mais próximo
T6 (21.10)	13.10
...	...

Vizinho mais próximo: T3. Te1 recebe **Age = 26** (o valor de T3).

Tratamento de dados ausentes

- **Cenário A (errado): fit no treino + teste juntos**
- Agora o "pool de vizinhos" inclui **Te2** também. Recalculando:

Treino (6 passageiros, tudo conhecido):

ID	Age	Fare	Survived
T1	22	7.20	0
T2	38	71.30	1
T3	26	7.90	1
T4	35	53.10	1
T5	54	51.90	0
T6	2	21.10	1

Teste (2 passageiros — dados "nunca vistos"):

ID	Age	Fare	Survived
Te1	**NaN**	8.00	0
Te2	45	8.05	1

Vizinho candidato	|Fare − 8.00|
T3 (7.90)	0.10
Te2 (8.05)	**0.05** ← agora é o mais próximo!

Vizinho mais próximo: Te2 (outra instância do próprio teste!). Te1 recebe **Age = 45**.

Tratamento de dados ausentes

- **O que aconteceu**

- No Cenário A, o valor imputado para **Te1** (26 → 45) mudou **só porque Te2 estava no mesmo lote de avaliação**.

- Isso é o vazamento: em produção, quando Te1 chegar como um passageiro novo e real, o sistema não vai ter magicamente acesso a "Te2" para usar como vizinho, cada passageiro novo chega sozinho, sem outros casos de teste disponíveis ao lado dele.

- O Cenário B reflete essa realidade; o Cenário A, não.

Resumindo:

Se você deseja eliminar dados ausentes, o que pode ser feito?

- Remoção de objetos em valores ausentes em qualquer atributo preditivo
- Remoção de objetos em valores ausentes em todos os atributo preditivos
- Remoção de objetos com valor ausente em qualquer/todos os atributos preditivos selecionados
- Remoção de atributo preditivo com valor ausente em qualquer objeto
- Remoção de atributo preditivo com valor ausente em todos os objetos
- Remoção de atributo preditivo com valor ausente em um número determinado de objetos

Mas como fazer isso em Python?

- Veja o arquivo que está no CANVAS sobre o **Titanic**

AD_RF_Titanic_processamento.ipynb

Mais informações sobre Dados ausentes

- Se desejarem saber mais informações sobre **dados ausentes**, veja a tese do meu aluno de doutorado:

- **Tese**: MALTA: Um método para análise e tratamento de dados ausentes

  Fábio Martins de Oliveira

  Link: https://bib.pucminas.br/teses/Informatica_FabioMartinsDeOliveira_30652_TextoCompleto.pdf

Referências:

Veja o capítulo 3 do livro de IA abaixo.



